BPUFuzzer: Effective Fuzz Testing for Branching Transient Execution Vulnerabilities of RISC-V CPU

Rihui Sun[1], Jin Wu[1], Hanyin Liu[1], Zikang Tao[1], Gang Qu[3], Dongsheng Wang[2], Yongqiang Lyu[2], Jian Dong[1*]
[1]Harbin Institute of Technology, Harbin, Heilongjiang, China
[2]Tsinghua University, Beijing, China
[3]University of Maryland, College Park, MD, US

2025 62nd ACM/IEEE Design Automation Conference (DAC) | 979-8-3315-0304-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/DAC63849.2025.11133085

***Abstract*—This paper presents BPUFuzzer, a fuzz testing tool for detecting branching transient execution vulnerabilities in CPU RTL design. BPUFuzzer addresses two key challenges: generating testcases that capture complex control flows, and extracting essential data from vast hardware states to guide testcase selection. Utilizing a control flow graph-based testcase generation strategy with anomaly detection and employing fitness and coverage metrics, BPUFuzzer works on testcases that cover broader program flows and deliberately selects testcases to discover transient execution vulnerabilities effectively. When applied on RISC-V Boom v3, BPUFuzzer uncovered more Spectre types than the state-of-the-arts, including a previously unidentified variant, named Spectre-LOOP.**

I. Introduction

Transient execution vulnerabilities are a significant hardware security issue in modern CPUs. Attacks like Spectre [1] and Meltdown [2], along with their variants [3]–[11], severely impact the reliability of processor hardware. The majority of these vulnerabilities are caused by the malicious exploitation of the branch prediction unit (BPU), which is a commonly implemented microarchitecture in modern processors designed to optimize pipeline performance by predicting control flow changes thus preventing pipeline stalls [12]. The discovery of these vulnerabilities and their variants has raised new concerns about processor security, as these microarchitecture-based issues cannot be easily fixed by vendors and pose serious threats to information system security.

Recent advancements have been made in the automated detection of transient execution vulnerabilities. Several approaches [13]–[17] focus on testing limited code snippets based on prior knowledge, without addressing a broad range of code patterns. SpecDoctor [18] generates testcases without prior knowledge but imposes constraints that prevent loop generation in its implementation. Furthermore, it lacks coverage feedback mechanisms specific to transient execution vulnerabilities, limiting its ability to detect issues arising from complex hardware states. Moreover, current techniques limit their vulnerabilities detection without in-depth analysis of complex BPU behaviors. Detecting vulnerabilities within BPU poses two significant challenges. First, since the combination of branch instructions that can change the BPU state is very diverse, covering a broad range of control flow patterns is difficult. Second, it is challenging to efficiently search for testcases that tend to trigger transient execution from numerous testcase patterns corresponding to complex BPU states.

In this paper, we present BPUFuzzer, a pre-silicon fuzz testing tool designed to address these challenges and effectively detect branching transient execution vulnerabilities. In order to capture complex control flow patterns, we propose an effective testcase generation and validation method based on control flow graph (CFG). In addition, we propose effective fitness and coverage functions to guide testcase selection, innovatively taking into account the states of the Reorder Buffer (RoB) and the Branch Prediction Unit (BPU). This approach extends fitness and coverage metrics to the microarchitectural component level, improving vulnerability detection efficiency. Through BPUFuzzer testing, we identified a new transient execution pattern, termed the loop pattern, which can manifest as a new Spectre variant called Spectre-Loop. The main contributions of our work are:

- We propose a CFG-based testcase generation that encompasses all possible program control flows, including loop types that were not addressed in previous work.
- We propose an effective feedback method on fuzz testing to guide testcase selection. It significantly enhances the efficiency of uncovering transient execution vulnerabilities in complex hardware designs.
- The proposed fuzz tool successfully identified a new Spectre variant called Spectre-loop on Boom v3 CPU. Experiments also demonstrate that BPUFuzzer uncovers more Spectre types than previous work.

II. Related Work

The automated detection of transient execution vulnerabilities has gained significant attention. Prior work, such as Medusa [13], uses authenticated PoC snippets as test seeds. SpecFuzz [14] simulates branch misprediction in security-sensitive libraries. Absynthe [15] and Osiris [16] focus on finding specific side-channels. These approaches mainly test limited code snippets based on prior knowledge, rather than covering a broad range of code patterns and hardware states. Furthermore, these frameworks test only on post-silicon CPUs, limiting their ability to identify potential transient execution leaks in the pre-silicon design phase before the processor is taped out. In pre-silicon RTL-stage research, Introspectre [17] uses predefined code gadgets with a focus on meltdown-type vulnerabilities. DifuzzRTL [19] and SIGFuzz [20] apply a

*Corresponding author: dan@hit.edu.cn

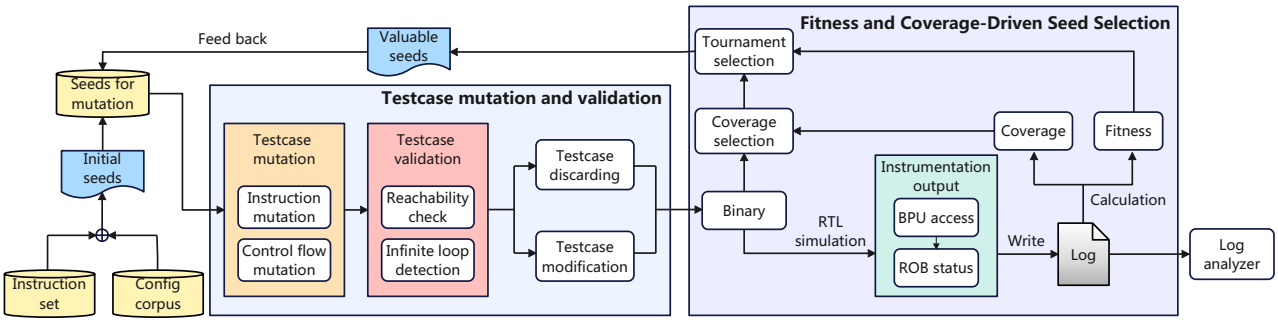


Fig. 1: Overview of BPUFuzzer

coverage based on control signals that drive the muxes to find CPU bugs and side channels. They focus on register states and ignore microarchitecture states, which is essential for transient execution. So their coverage definitions are unsuitable for detecting transient execution vulnerabilities. WhisperFuzz [21] focuses on timing behaviors changing by triggering different microarchitectural state transitions. In contrast, BPUFuzzer in-depth studies the states of microarchitectural components, including BPU and RoB. The coverage metrics formed by these states are used to search for testcases tend to trigger branching transient execution, significantly improving the efficiency of detecting vulnerabilities. SpecDoctor [18] generates testcases using CFG construction. To avoid invalid testcases in implementation, it enforces constraints that prevent loop generation. However, loop is an important control flow transfer pattern. Moreover, it does not provide coverage feedback mechanisms, limits its detection from complex hardware states. In contrast, BPUFuzzer is capable of discovering transient execution caused by any control flow transfer. Moreover, BPUFuzzer tailored coverage and fitness metrics optimized for transient execution. Owing to these two improvements, BPUFuzzer surpasses previous work in both the scope and efficiency of detecting transient execution vulnerabilities.

III. Overview of BPUFuzzer

In this section, we present an overview of the architecture of our transient execution vulnerability fuzz testing tool, BPUFuzzer. We consider the following threat model which aligns with the mainstream studies in discovering transient execution vulnerabilities such as Spectre and Meltdown:

(1) The attacker is a user without privilege, the victim is the privileged kernel. The attacker and the victim locate on the same logical core;

(2) The attacker can invoke the victim through system calls.

As shown in Figure 1, our tool includes two main modules to address the challenges outlined in Section I. The testcase mutation and validation module generates testcases for fuzz testing. To maximize the range of control flow patterns in testcase generation, We employ a modified CFG-based technique that includes instruction mutations and control flow mutations without any pattern constraints. A testcase validation submodule is also included to detect and avoid infinite loops. These methods enable us to produce a greater variety of control flow patterns compared with state-of-the-art tools.

The fitness and coverage-driven seed selection module select testcases with high tendency of transient execution and feed them back as seeds for mutation. Aimed at selecting valuable testcases efficiently, BPUFuzzer incorporates hardware information through instrumentation, including the status of BPU and RoB, which is closely related to transient execution. These pieces of information are logged and used to calculate the fitness function and testcase coverage. Specifically, testcases are first selected based on whether they cover new hardware states, as determined by coverage calculation, and then undergo tournament selection based on fitness value. Additionally, a log analyzer detects transient execution risks through the collected data and classifies them based on the originating microarchitectural component.

Sections IV and V will detail the working principles of the testcase mutation and validation module, as well as the fitness and coverage-driven seed selection module.

IV. Testcase Mutation and Validation

The testcase mutation and validation module aims to generate testcases that cover the maximum possible range of control flow patterns. This module consists of two components: mutation of instructions within a basic block and mutation of control flow instructions. To ensure that generated testcases maximize control flow coverage and execute to termination, a validation module checks each testcase after mutation.

A. Mutation Rule of a Basic Block

Figure 2 shows the structure of a basic block (BB) and all possible mutation behaviors. A BB must end with either a conditional or unconditional jump instruction, with no jump instructions allowed at any other position within the block. During mutation, various transformations may occur within the BB, including the insertion, removal, substitution of instructions, and modifications to jump instruction targets.

B. Control Flow Related Instructions on RISC-V

In RISC-V instruction sets, there are two types of instructions related to control flow transfer, unconditional jump instructions and conditional branch instructions, shown in Table I. A conditional branch adds two outgoing edges to the basic block in the CFG, while an unconditional jump only adds one outgoing edge.

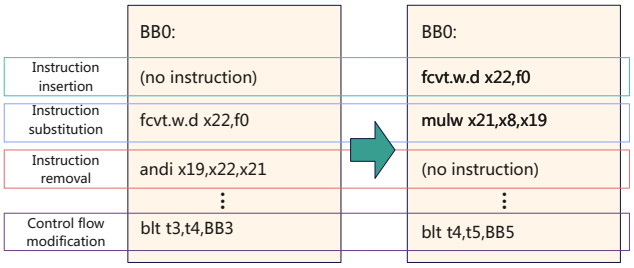


Fig. 2: Possible mutation operations on a basic block

TABLE I: Control Flow Related RISC-V Instructions

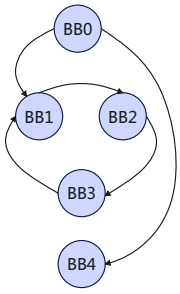
Inst	Description
beq, rs1, rs2, imm	if(rs1 == rs2) PC += imm
bne, rs1, rs2, imm	if(rs1 != rs2) PC += imm
blt, rs1, rs2, imm	if(rs1 $<$ rs2) PC += imm
bge, rs1, rs2, imm	if(rs1 $\geq$ rs2) PC += imm
bltu, rs1, rs2, imm	if(rs1 $<$ rs2) PC += imm
bgeu, rs1, rs2, imm	if(rs1 $\geq$ rs2) PC += imm
jal, rd, imm	rd = PC+4; PC += imm
jalr, rd, rs1, imm	rd = PC+4; PC = rs1 + imm

C. Testcase Validation

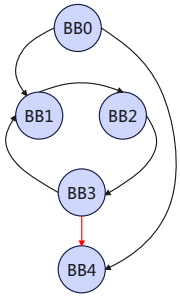
To maximize coverage of flow transfer scenarios in CFG generation, we do not restrict jump or branch targets during mutation. However, arbitrary jump combinations may create infinite loops, so we apply a validation process to detect and correct invalid testcases.

All loops in a CFG have the potential to cause infinite loops. Based on the types of jumps at the end of the BBs of a loop, we classify the causes of infinite loops into the following categories:

- **Unconditional jumps**: In this type, all BBs within the loop end with unconditional jumps. Consequently, once the control flow enters such a loop, it becomes impossible to exit. An example of this situation is illustrated in Figure 3(a), once the control flow jumps from BB0 to BB1, it will never exit. We will refer to this type of testcase as type 1 testcase in the following discussion.



(a) Infinite loop caused by unconditional jumps



(b) Infinite loop caused by conditional branches that always branch in the same direction

Fig. 3: Two types of invalid testcases

- **Conditional jumps with fixed direction**: Although at least one BB within these loops end with conditional jumps that could theoretically allow for an exit, arbitrary mutation operations may cause the direction of the conditional jump to remain unchanged, resulting in an infinite loop. As shown in Figure 3(b), although theoretically the unconditional jump at the end of BB3 could jump to BB4, improper instruction mutation may causes the value comparison between the two source registers of BB3's conditional jump to remain unchanged, preventing the jump from BB3 to BB4 from ever occurring(illustrated with a red arrow) and thus forming an infinite loop. We will refer to this type as type 2 testcase.

Notation	Description
BB.jmpIns	Last jump instruction of basic block `BB`.
BB.jmpIns.name	Name of the instruction.
BB.jmpIns.rs1	Source register 1 of the jump instruction.
BB.jmpIns.rs2	Source register 2 of the jump instruction.
instruction.dest	Destination register of arithmetic & logical instructions.

TABLE II: Notations used in Algorithm 1.

Algorithm 1: Type 2 testcases modification

```
Input : BBs containing conditional jump instructions in a cycle
Output: Modified BBs
foreach BB in BBs do
    if BB.jmpIns.name in {blt, bge, bltu, bgeu} then
        Set BB.jmpIns.rs1 to t1;
        Set BB.jmpIns.rs2 to t2;
        foreach instruction in BB do
            if instruction.dest in {t1, t2} then
                Delete instruction;
        ▷ Adjust the two source registers according to their current states;
        Insert instruction slti t3, t1, t2;
        Insert instruction xor t4, t3, 1;
        Insert instruction slli t3, t3, 6;
        Insert instruction slli t4, t4, 6;
        Insert instruction add t1, t1, t3;
        Insert instruction add t2, t2, t4;
        Insert instruction addi t1, t1, -32;
        Insert instruction addi t2, t2, -32;
    if BB.jmpIns.name in {beq, bne} then
        Set BB.jmpIns.rs1 to zero;
        Set BB.jmpIns.rs2 to t2;
        foreach instruction in BB do
            if instruction.dest = t2 then
                Delete instruction;
        Insert instruction srli t2, 1;
```

To identify and correct the two types of invalid testcases mentioned above, we employed a two-phase detection and modification approach after each round of mutation.

- **Discarding type 1 testcases**: We verify that all basic blocks reachable from the starting BB can also reach the terminating BB, any testcase that fails to meet this criterion will be discarded. Thus the type 1 invalid testcase can be avoided.

- **Modifying type 2 testcases**: After all type 1 testcases are discarded, each rest cycle in generated CFGs contains at least one conditional jump. In order to allow the direction of conditional jumps to change, we implement the following two measures: (I) Inserting instructions to adjust the two source registers according to their current states. For blt, bge, $bltu$ and $bgeu$, if the value of register a exceeds that of register b, we add instructions to reduce the value in register a and increase the value in register b, and vice versa. For bne and beq, since the probability of two 32-bit numbers being equal is extremely low, we implemented a specific approach: one source register is set to special zero register defined in RV32I [22], which always holds the value zero, while the other register is initialized with a non-zero value. Additionally, we insert a logical right shift instruction, consequently, the values in the two registers are always unequal initially, but after a finite number of loops, they will become equal. (II) Ensuring that no other instructions within the loop modify the contents of registers related to the jump. We deleted all instructions modifying any of the two resource registers before we insert the above instructions. The implementation can be found in Algorithm 1. The notations used in it are defined in Table II.

V. Fitness and Coverage-Driven Seed Selection

In this section, we present our implementation of the coverage and fitness functions and how they enhance our fuzz tool's ability to select more effective testcases.

A. Hardware Instrumentation

We collect essential microarchitectural data during the hardware instrumentation phase. The data that is interested is collected based on our study of the branching transient execution. The specific microarchitectural details are listed in Table III. We primarily gather information on BPU access and RoB status, which clearly reveals the behavior of transient execution and distinguishes them from normal program execution.

TABLE III: Hardware instrumentation information summary

uArch Comp.	Instrumentation Output	Description
RoB	mispredict_pc	The program counter of the mispredicted branch instruction.
	mispredict_ins_code	The instruction code of the mispredicted branch instruction.
	mispredict_target_addr	Mispredicted target address.
	mispredict_count	Total count of mispredictions.
BTB	wbtb_clock	Clock count when writing BTB.
	wbtb_bank_sel	Selection of bank when writing BTB, part of BTB index rule.
	wbtb_meta_bank_sel	Selection of bank when writing BTB meta data(tag and type), part of BTB index rule.
	wbtb_way	Selection of way when writing BTB, part of BTB index rule.
	wbtb_set	Selection of set when writing BTB, part of BTB index rule.
	wbtb_branch_offset	Offset of the branch ,i.e.target_pc-current_pc
	wbtb_branch_type	Type of branch instruction, i.e.conditional/unconditional
	wbtb_branch_tag	Tag associated with the branch, i.e. part of PC
	btb_hit_count	Count of BTB hits.
	btb_hit_set	The hit BTB set.
uBTB	ubtb_hit_count	Count of uBTB hits.
	ubtb_hit_tag	Tag in hit uBTB entry.
LOOP	loop_hit_count	Count of LOOP hits.
	loop_hit_set	Set of hit LOOP entry.
TAGE	tage_hit_count	Count of TAGE hits.
	tage_hit_table_idx	Index of hit TAGE table.
	tage_counter_count	Count of TAGE 3-bit counter.

B. Design of Fitness Function and Coverage

Using the data in Table III, we designed a fitness function to estimate the likelihood of a testcase mutating into one that triggers transient execution. Additionally, we developed a coverage metric to efficiently search for testcases that tend to trigger transient execution from numerous testcase patterns corresponding to complex BPU states.

The formula of the fitness function is as follows:

$$fitness = mispredict\_count \times weight + btb\_hit\_count \quad (1)$$

The fitness function is a weighted sum of the misprediction count in the RoB and the BTB hit count. The misprediction count plays a critical role in triggering transient execution, while the BTB hit count is associated with the attacker's training phase in vulnerabilities like Spectre and its variants. In our testcase selection, greater emphasis is placed on the misprediction count, which is assigned a larger weight.

For coverage, since each control flow transfer affects BTB states, we incorporate several branch prediction-related values into our coverage metric, which is shown in Algorithm 2.

In transient execution exploitation, the prediction training and triggering rely on the branch/jump instructions with specific instruction addresses, jump directions and target addresses. These values are stored in BTB and BTB meta table. To analyze these behaviors, we extract the relevant information and hash each corresponding piece of data in the log to generate binary feature values ($t.btb$, $t.btb\_meta$). $t.btb$ represents the features of jump directions and target addresses, while $t.btb\_meta$ represents the features of instruction addresses. If there is a new feature value, there is a control flow transfer instruction whose address or target address has changed. It may bring new transient execution possibility, reflected in an increase in test coverage.

Based on our fitness function and coverage, we select testcases to enhance defect detection effectiveness. First, we prioritize testcases that generate new coverage. If a testcase

Algorithm 2: Coverage Calculation

Input : A set of testcase and their logs
Output: whether the testcase is a new coverage

```
cov_list = empty set;
cov_sum = 0;
foreach t in test_case_set do
    t.btb = 0;
    t.btb_meta = 0;
    foreach line in test_case.log do
        btb_info = wbtb_set + wbtb_way +
          wbtb_branch_offset + wbtb_bank_sel;
        meta_info = wbtb_set + wbtb_way +
          wbtb_branch_tag + wbtb_branch_type +
          wbtb_meta_bank_sel;
        t.btb = t.btb xor btb_info;
        t.btb_meta = t.btb_meta xor meta_info;
    if (t.btb,t.btb_meta) not in cov_list then
        cov_list.insert(t.btb,t.btb_meta);
        t.coverage = true;
        cov_sum += 1;
    else
        t.coverage = false;
```

generates new coverage, it is added to the candidate set, while those without new coverage have a 5% probability of being included to prevent local optima. All testcases in the candidate set then undergo a tournament selection based on the fitness to form the next generation of testcases. This approach ensures that testing progressively covers more hardware states and that testcases increasingly tend to produce transient execution. The discovery of a new Spectre variant, called Spectre-Loop (detailed in Section VII), is owed to this methodology.

VI. Evaluation of BPUFuzzer

In this section, we present experiments that demonstrate the advantage of our proposed BPUFuzzer in terms of testcase generation pattern, coverage and vulnerability detection. We compare them with existing state-of-the-art tool, SpecDoctor.

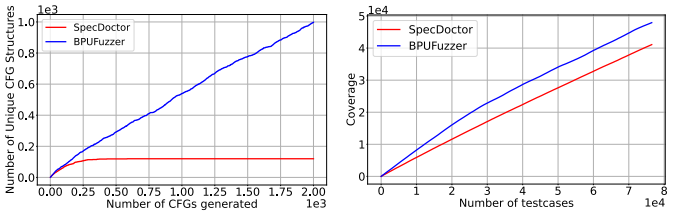
A. Experiment Setup

RISC-V CPU under test. We evaluated BPUFuzzer on the open-source out-of-order RISC-V CPU: Boom V3 [23]. It supports RV64G ISA which includes supervisor level instructions and user level instructions. It contains the out-of-order microarchitectural units including RoB and branch predictors.

Fuzz testing environment. We evaluated BPUFuzzer with a RTL simulator called Chipyard [24]. It is a modified Verilator simulator designed for Boom RISC-V CPU. All the fuzzing experiments are conducted on a machine equipped with an Intel(R) Xeon(R) E5-2670 v3 CPU at 2.30GHz with 64GB of RAM, which runs Ubuntu 22.04 LTS.

B. Testcase Generation Pattern

In section IV, we propose a testcase mutation and validation method designed to cover diverse control flow transfer patterns, corresponding to mutually non-isomorphic control flow graphs [25]. We verify the number of patterns generated and compare it with SpecDoctor.



(a) comparison of testcase pattern with SpecDoctor (b) comparison of test coverage with SpecDoctor

Fig. 4: Comparison with SpecDoctor

In Figure 4(a), we applied a basic block limit of 6 in the testcase generation process for both our tool and SpecDoctor. In SpecDoctor, basic blocks are connected by conditional jump instructions, and each block can only link to subsequent blocks, preventing loop structures. Under these constraints, SpecDoctor can generate only 120 unique testcase patterns, equivalent to 5 factorial. In contrast, our tool produces a greater variety of control flow transfer patterns, including testcases with loops that execute successfully.

C. Coverage

In Section V, we introduced a fitness function and coverage design based on microarchitectural state to guide testcase selection and mutation, and conducted a comparative experiment with SpecDoctor in Figure 4(b). Using the same hardware setup and the same testcase limit of 76,000, we tested on both tools respectively. SpecDoctor generated 34,916 testcases that do not generate new hardware states, which is unprofitable for transient execution detection. BPUFuzzer covered 6,846 more hardware states than SpecDoctor, leading by 16.7% in coverage. We also ran BPUFuzzer with and without coverage feedback under the same hardware setup for 40 hours. With coverage feedback, BPUFuzzer identified an average of 12.89 testcases with transient execution risks per minute, compared to 11.26 testcases per minute without coverage feedback. This represents a 14.5% improvement in transient execution detection efficiency, highlighting the benefits of coverage feedback.

D. Vulnerability Detection

We ran BPUFuzzer and SpecDoctor under the same hardware setup for a week and classified found vulnerability types following previous research [12]. The found vulnerability types comparison is shown in Table IV. BPUFuzzer successfully detected transient execution in micro BTB and LOOP while SpecDoctor do not.

In Boom v3, micro BTB is a prediction unit that stores local target addresses. It utilizes a fully-associative index method and only works when the corresponding BTB entry is empty. Spectre_uBTB needs to train the micro BTB while clearing the BTB entries. It requires the precise coordination of branch instructions in a wider range of instruction addresses than Spectre_BTB. However, the range of instruction addresses is relatively limited in SpecDoctor, makes it incapable of generating this vulnerability type.

TABLE IV: Found vulnerabilities summary and comparison with SpecDoctor

Category	Name	Description	BPUFuzzer	SpecDoctor
Bound Check Bypass	Spectre_BIM_IP	Exploitation of Bi-Modal Table.	✓	✓
	Spectre_uBTB_IP	Exploitation of micro BTB.	✓	×
	Spectre_TAGE_IP	Exploitation of TAGE.	✓	✓
	Spectre_TAGE_OP	Exploitation of TAGE with Table entry collision.	✓	✓
	Spectre_Loop_IP	Exploitation of Loop.	✓	×
	Spectre_Loop_OP	Exploitation of Loop with Table entry collision.	✓	×
Branch Target Injection	Spectre_uBTB_IP	Exploitation of micro BTB.	✓	×
	Spectre_BTB_IP	Exploitation of BTB.	✓	✓

VII. New Finding of BPUFuzzer: Spectre-Loop

During our analyzing on experiment results, we found some vulnerabilities have significantly large btb_hit_count in their fitness value. Through our further analysis, We found that they all have a loop structure. We eventually confirmed that they exploited a loop prediction optimization mechanism in Boom v3 and formed a new Spectre variant, termed Spectre-Loop.

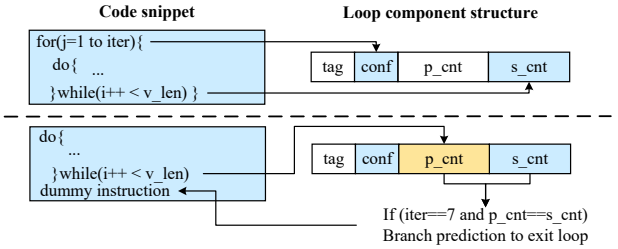
A. Details of Spectre-Loop

Figure 5(a) describes the working mechanism of LOOP, which predicts long loop exit timing of up to 1024. LOOP table has 4 parts: tag derived from the instruction's PC, is used to identify matching entries in the table. $iter$ is a three-bit saturating counter records times the loop is executed as a whole. p_cnt represents the present count of times the loop body has entered the pipeline during loop iteration, and it is updated each time the loop body enters the pipeline. s_cnt records the sum number of iterations after the loop code has fully completed execution. When the loop code runs again, if $iter$ reaches the threshold and p_cnt equals s_cnt, the prediction will indicate a loop exit. In summary, It is a prediction mechanism triggered by a loop code structure, this control flow pattern is not included in the testcase generation implementation of SpecDoctor. Thus SpecDoctor unable to detect transient execution caused by this prediction mechanism.

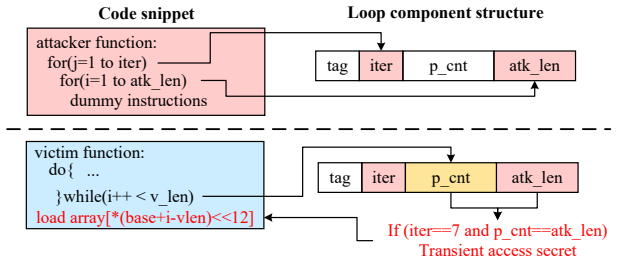
Figure 5(b) demonstrates the attack flow of spectre-loop. First, in the attacker phase, the loop is executed multiple times to train the s_cnt to the value atk_len. Then, in the trigger phase, the loop predict loop exit early after executing atk_len times. During speculative execution, this results in a transient out-of-bounds access on the offset of $atk\_len - v\_len$ (with $atk\_len < v\_len$). By controlling atk_len, an attacker can bypass the bound check and transiently access arbitrary locations, enabling access to specific secret data. The attacker can subsequently recover the transiently accessed secret data using a simple flush+reload technique.

B. Analysis of Spectre-Loop

Spectre-Loop successfully retrieved the secret of 1,024 bits with the error rate under 0.01%. Each bit leakage cost 220k clock cycles on the Chipyard simulator. The leakage rate is 4.9 Kb/s assuming a CPU clock rate of 1GHz.



(a) loop prediction optimization mechanism



(b) attack scenario of spectre-loop

Fig. 5: Loop prediction mechanism & Process of Spectre-loop

VIII. Conclusion

In this paper, we propose a pre-silicon fuzz testing tool, BPUFuzzer, to detect transient execution vulnerabilities. We propose a novel modified CFG-based test case generation method that outperforms all existing approaches in generating a broader range of control flow patterns, particularly by incorporating loop patterns that have not been considered in any prior work. Additionally, We develop novel coverage and fitness metrics tailored to the characteristics of transient execution, significantly enhancing the efficiency in detecting transient execution vulnerabilities. Finally, BPUFuzzer finds a newly discovered Spectre variant, termed Spectre-Loop, which exploits a previously overlooked prediction mechanism to perform a Bounds Check Bypass attack.

Acknowledgment

This work was supported by the Fundamental Research Funds for the Central Universities (Grant No. HIT.OCEF.2024012) and the National Natural Science Foundation of China (Grant No. U23B2041, U24A6009).

References

[1] P. Kocher, J. Horn, A. Fogh, D. Genkin, D. Gruss, W. Haas, M. Hamburg, M. Lipp, S. Mangard, T. Prescher, M. Schwarz, and Y. Yarom, "Spectre attacks: Exploiting speculative execution," in *2019 IEEE Symposium on Security and Privacy, SP 2019, San Francisco, CA, USA, May 19-23, 2019*. IEEE, 2019, pp. 1–19.

[2] M. Lipp, M. Schwarz, D. Gruss, T. Prescher, W. Haas, A. Fogh, J. Horn, S. Mangard, P. Kocher, D. Genkin, Y. Yarom, and M. Hamburg, "Meltdown: Reading kernel memory from user space," in *27th USENIX Security Symposium (USENIX Security 18)*. Baltimore, MD: USENIX Association, Aug. 2018, pp. 973–990.

[3] E. M. Koruyeh, K. N. Khasawneh, C. Song, and N. B. Abu-Ghazaleh, "Spectre returns! speculation attacks using the return stack buffer," in *12th USENIX Workshop on Offensive Technologies, WOOT 2018, Baltimore, MD, USA, August 13-14, 2018*, C. Rossow and Y. Younan, Eds. USENIX Association, 2018.

[4] J. Wikner and K. Razavi, "RETBLEED: arbitrary speculative code execution with return instructions," in *31st USENIX Security Symposium, USENIX Security 2022, Boston, MA, USA, August 10-12, 2022*, K. R. B. Butler and K. Thomas, Eds. USENIX Association, 2022, pp. 3825–3842.

[5] H. Yavarzadeh, A. Agarwal, M. Christman, C. Garman, D. Genkin, A. Kwong, D. Moghimi, D. Stefan, K. Taram, and D. M. Tullsen, "Pathfinder: High-resolution control-flow attacks exploiting the conditional branch predictor," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3, ASPLOS 2024, La Jolla, CA, USA, 27 April 2024- 1 May 2024*, R. Gupta, N. B. Abu-Ghazaleh, M. Musuvathi, and D. Tsafrir, Eds. ACM, 2024, pp. 770–784.

[6] S. van Schaik, A. Milburn, S. Österlund, P. Frigo, G. Maisuradze, K. Razavi, H. Bos, and C. Giuffrida, "RIDL: rogue in-flight data load," in *2019 IEEE Symposium on Security and Privacy, SP 2019, San Francisco, CA, USA, May 19-23, 2019*. IEEE, 2019, pp. 88–105.

[7] M. Schwarz, M. Lipp, D. Moghimi, J. V. Bulck, J. Stecklina, T. Prescher, and D. Gruss, "Zombieload: Cross-privilege-boundary data sampling," in *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, CCS 2019, London, UK, November 11-15, 2019*, L. Cavallaro, J. Kinder, X. Wang, and J. Katz, Eds. ACM, 2019, pp. 753–768.

[8] J. Van Bulck, D. Moghimi, M. Schwarz, M. Lippi, M. Minkin, D. Genkin, Y. Yarom, B. Sunar, D. Gruss, and F. Piessens, "Lvi: Hijacking transient execution through microarchitectural load value injection," in *2020 IEEE Symposium on Security and Privacy (SP)*, 2020, pp. 54–72.

[9] G. Maisuradze and C. Rossow, "ret2spec: Speculative execution using return stack buffers," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '18. New York, NY, USA: Association for Computing Machinery, 2018, p. 2109–2122.

[10] E. Barberis, P. Frigo, M. Muench, H. Bos, and C. Giuffrida, "Branch history injection: On the effectiveness of hardware mitigations against Cross-Privilege spectre-v2 attacks," in *31st USENIX Security Symposium (USENIX Security 22)*. Boston, MA: USENIX Association, Aug. 2022, pp. 971–988.

[11] H. Ragab, E. Barberis, H. Bos, and C. Giuffrida, "Rage against the machine clear: A systematic analysis of machine clears and their implications for transient execution attacks," in *30th USENIX Security Symposium, USENIX Security 2021, August 11-13, 2021*, M. D. Bailey and R. Greenstadt, Eds. USENIX Association, 2021, pp. 1451–1468.

[12] C. Canella, J. V. Bulck, M. Schwarz, M. Lipp, B. von Berg, P. Ortner, F. Piessens, D. Evtyushkin, and D. Gruss, "A systematic evaluation of transient execution attacks and defenses," in *28th USENIX Security Symposium (USENIX Security 19)*. Santa Clara, CA: USENIX Association, Aug. 2019, pp. 249–266.

[13] D. Moghimi, M. Lipp, B. Sunar, and M. Schwarz, "Medusa: Microarchitectural data leakage via automated attack synthesis," in *29th USENIX Security Symposium (USENIX Security 20)*. USENIX Association, Aug. 2020, pp. 1427–1444.

[14] O. Oleksenko, B. Trach, M. Silberstein, and C. Fetzer, "SpecFuzz: Bringing spectre-type vulnerabilities to the surface," in *29th USENIX Security Symposium (USENIX Security 20)*. USENIX Association, Aug. 2020, pp. 1481–1498.

[15] B. Gras, C. Giuffrida, M. Kurth, H. Bos, and K. Razavi, "Absynthe: Automatic blackbox side-channel synthesis on commodity microarchitectures," in *27th Annual Network and Distributed System Security Symposium, NDSS 2020, San Diego, California, USA, February 23-26, 2020*. The Internet Society, 2020.

[16] D. Weber, A. Ibrahim, H. Nemati, M. Schwarz, and C. Rossow, "Osiris: Automated discovery of microarchitectural side channels," in *30th USENIX Security Symposium (USENIX Security 21)*. USENIX Association, Aug. 2021, pp. 1415–1432. [Online]. Available: https://www.usenix.org/conference/usenixsecurity21/presentation/weber

[17] M. Ghaniyoun, K. Barber, Y. Zhang, and R. Teodorescu, "INTROSPECTRE: A pre-silicon framework for discovery and analysis of transient execution vulnerabilities," in *48th ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2021, Virtual Event / Valencia, Spain, June 14-18, 2021*. IEEE, 2021, pp. 874–887.

[18] J. Hur, S. Song, S. Kim, and B. Lee, "Specdoctor: Differential fuzz testing to find transient execution vulnerabilities," in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 1473–1487.

[19] J. Hur, S. Song, D. Kwon, E. Baek, J. Kim, and B. Lee, "Difuzzrtl: Differential fuzz testing to find cpu bugs," in *2021 IEEE Symposium on Security and Privacy (SP)*, 2021, pp. 1286–1303.

[20] C. Rajapaksha, L. Delshadtehrani, M. Egele, and A. Joshi, "Sigfuzz: A framework for discovering microarchitectural timing side channels," in *2023 Design, Automation Test in Europe Conference Exhibition (DATE)*, 2023, pp. 1–6.

[21] P. Borkar, C. Chen, M. Rostami, N. Singh, R. Kande, A. Sadeghi, C. Rebeiro, and J. Rajendran, "Whisperfuzz: White-box fuzzing for detecting and locating timing vulnerabilities in processors," in *33rd USENIX Security Symposium, USENIX Security 2024, Philadelphia, PA, USA, August 14-16, 2024*, D. Balzarotti and W. Xu, Eds. USENIX Association, 2024.

[22] A. Waterman and K. Asanović, "The risc-v instruction set manual, volume i: User-level isa, document version 20191214-draft," *RISC-V Foundation*, December 2019.

[23] J. Zhao, B. Korpan, A. Gonzalez, and K. Asanovic, "Sonicboom: The 3rd generation berkeley out-of-order machine," May 2020.

[24] A. Amid, D. Biancolin, A. Gonzalez, D. Grubb, S. Karandikar, H. Liew, A. Magyar, H. Mao, A. Ou, N. Pemberton, P. Rigge, C. Schmidt, J. Wright, J. Zhao, Y. S. Shao, K. Asanović, and B. Nikolić, "Chipyard: Integrated design, simulation, and implementation framework for custom socs," *IEEE Micro*, vol. 40, no. 4, pp. 10–21, 2020.

[25] L. Babai, "Group, graphs, algorithms: the graph isomorphism problem," in *Proceedings of the International Congress of Mathematicians: Rio de Janeiro 2018*. World Scientific, 2018, pp. 3319–3336.

Synthesis of CFET Cell Library Leveraging Backside Metal Routing

Ting-Xin Lin, Yih-Lang Li
Department of Computer Science, National Yang Ming Chiao Tung University, Hsinchu, Taiwan
jack25946705@gmail.com, ylli@cs.nycu.edu.tw

2025 62nd ACM/IEEE Design Automation Conference (DAC) | 979-8-3315-0304-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/DAC63849.2025.11133087

ABSTRACT

As technology nodes continue to shrink, Complementary FET (CFET) structures, which stack PMOS and NMOS together, have emerged as a promising candidate for next-generation technology. Due to the reduction in routing tracks, the insertion of dummy polys to increase routing resources and the use of M2 during the synthesis of CFET standard cells have become more inevitable. These two factors make block-level routing significantly more challenging. To address this, we introduce the methods to utilize the backside (BS) routing resources at the CFET standard cell synthesis stage and efficiently handle CFET transistor folding. To the best of our knowledge, this is the first work to consider BS routing resources at the cell synthesis stage and efficiently address transistor folding in the CFET stacked structure. In the transistor folding and placement stages, we utilize Euler paths to estimate the lower bound of contacted poly pitch (CPP) and apply dynamic programming (DP) to calculate the frontside minimum required tracks (FMRT) of BS-resource-aware placements. The subsequent satisfiability modulo theories (SMT) approach determines which tracks the devices occupy and completes cell routing. Experimental results show that compared to previous work [11], we achieve reductions of 1%, 45%, and 19% in #CPP, #M2 tracks, and runtime, respectively.

1. INTRODUCTION

Due to the scaling of semiconductor technology, device structures like FinFET face limitations due to the separation between two types of devices. Therefore, CFET structures, which stack both device types together, have become a mainstream approach for the next generation technology. However, the reduced number of available routing tracks and complex design rules impose significant challenges on both cell-level and block-level routing in the frontside (FS) back end of line (BEOL). Backside power delivery networks (BS PDN) [1] have demonstrated IR-drop reduction in both 2D and 3D designs. [2] further introduces BS signal routing to enhance performance and power efficiency at the 2nm technology node. Inspired by [2], we apply BS routing resources, including nano-Through-Silicon Vias (nTSVs) and backside metals (BSMs), during the CFET cell synthesis stage to alleviate routing congestion on the FS of the chip. Fig. 1 illustrates the impact of utilizing BS resources. The signal *Y* is routed through M0 metals, nTSVs and BSMs (Fig. 1(a)) without the use of M1 and M2 wires, whereas in Fig. 1(b), two M1 wires and one M2 track are required to complete the routing. Compared to Fig. 1(b), Fig 1(a) saves two M1 wires and one M2 track.

In CFET structures, the contact allocation for the bottom device signal can be blocked by the top device signal in the same column when they differ and partially overlap in the *z*-direction, with each source, drain, or gate signal occupying a separate column. This issue becomes more severe when the top device obstructs more routing tracks, and it is evident that a larger top device blocks more tracks in a column. Transistor folding, which divides a larger transistor into multiple smaller

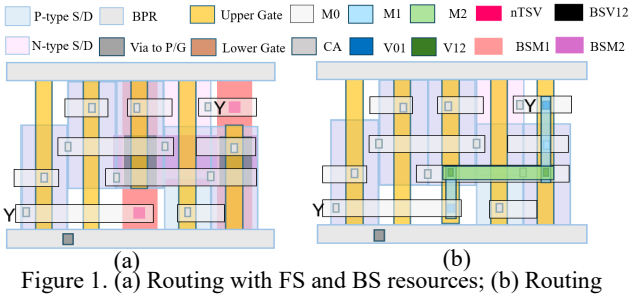


Figure 1. (a) Routing with FS and BS resources; (b) Routing with only FS resources.

Figure 2. (a) CFET folding; (b) Simple folding.

transistors (fingers), helps release more routing resources for the bottom device. However, transistor folding in CFET stacked structures is simplified, with previous works mainly focusing on planning structures. In the non-stacked structures, [3] addresses both transistor folding and placement simultaneously through search tree exploration. [4] applies ILP to determine the size of each finger and minimize cell area. [5] estimates the cell width using an Euler graph and presents search tree pruning techniques. The main difference of folding problems between in stacked and non-stacked structures is that, in stacked structures, we need to account for the space required to allocate contacts for bottom signals to access M0. Besides, measuring this space is critical, which is not a concern in non-stacked structures. Fig. 2 highlights the significance of the folding issue in CFET. In Fig. 2(a), the bottom device signal *S* is successfully routed via M0 through the diffusion contact access. However, as shown in Fig. 2(b), the contact location at the cross position for the bottom signal S fails due to being blocked by the top device, ultimately causing routing failure.

Previous works have addressed transistor placement with various approaches. For instance, [6] applies a bipartite graph and graph-based chaining processes to maximize diffusion sharing, while [7] uses DP to generate routability-aware transistor placement. For cell routing, common algorithms like maze, Boolean satisfiability (SAT), integer linear programming (ILP), mixed integer linear programming (MILP) and network flow are employed. The proposed ILP M0 planning model in [7] systematically selects M0 segments, followed by implicitly adjustable grid map maze routing to produce higher-quality cell layouts that are compliant with ASAP 7nm technology compared to the handcrafted cell library. The SAT approach in [8] generates cell layouts free of design-rule violations and minimizes undesired patterns while considering design-for-manufacturing. In [9], a spare M1 track is reserved to enhance

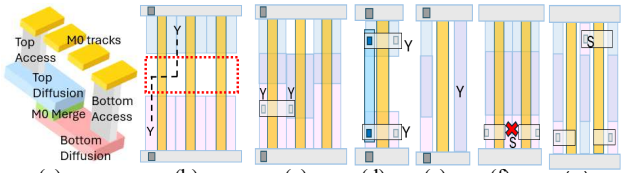


Figure 3. (a) A 3D view; (b) (c) FinFet and CFET structures; (d) With real MOS widths; (e) With simplified MOS widths; (f) Our routing model; (g) The simplified routing model

pin accessibility, and MILP is used to concurrently complete cell routing and pin allocation. In previous CFET cell library synthesis works, [10] simultaneously handles placement and routing using SMT. However, due to the high complexity of the model, generating large cells faces scalability challenges. Moreover, since placement and routing are explored simultaneously, the process may overlook placements that could lead to more efficient routing due to time constraints. The approach [11] demonstrates that traditional two-stage cell synthesis, cell placement and cell routing, can not only outperform [10] in terms of the cell area, but also additionally resolve the scalability issue. Nonetheless, these CFET works ignore transistor widths specified in SPICE netlists, which will be discussed in more detail in Section 2. Synthesized standard cell layouts must comply with SPICE netlists to ensure cell functionality and characteristics, such as performance, and power consumption.

This is the first work to introduce BS routing resources during the cell synthesis and to handle the folding issue for transistors with large width in CFET stacked structures. Our main contributions are as follows: (1) The CPP lower bound of a folding outcome is calculated using Euler paths, followed by a DP-based FMRT calculation of BS-resource-aware placements; (2) The proposed SMT formulation decides the tracks used by the devices and performs cell routing considering BS resources; and (3) Compared to previous work [11], we achieve reductions of 1%, 45%, and 19% in #CPP, #M2 tracks, and runtime, respectively.

In Section 2, we introduce the CFET cell structure and highlight the differences in the routing model between our approach and previous works. Section 3 addresses CFET folding, BS-resource-aware placement, and the SMT formulations for cell routing. Section 4 presents the experimental results and Section 5 concludes this work.

2. PRELIMINARY

2.1 CFET Stacked Structures

In CFET with buried power rails (BPRs), the middle region disappears due to stacking PMOS and NMOS together. The following two CFET routing features simplify connecting signals between two different types of devices, making it easier than in FinFET. First, a vertical wire connecting two diffusion signals, which overlap in the *z*-direction, is replaced by an M0-merge to interconnect them. Second, a Z-shaped wire can be eliminated by using a horizontal M0 wire to connect signals from both types of devices, provided their contact allocations are on the same routing track. Fig. 3(a) shows a 3D view of the M0-merge. The red dotted rectangle in Fig. 3(b) represents the middle region in FinFET structures, where the signal *Y* requires vertical wires. In contrast, Fig. 3(c) shows that no vertical wires are needed in CFET structures.

2.2 The Previous Routing Models

The previous work [11] simplifies both the routing model and MOS widths. There are at most two signals in a column. Since the two signals in a column are distinct and the bottom

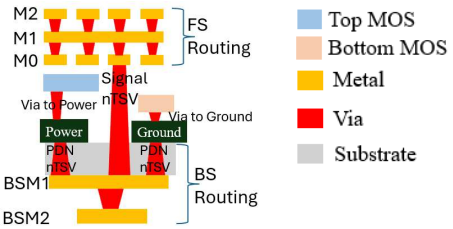


Figure 4. Layers on front and back sides.

signal requires a contact to access the upper M0 layer, contact placement for this signal is limited to either the top or bottom track to avoid the obstruction caused by the top device. However, if the top signal is VDD or VSS, and connects to power or ground rail, the bottom signal has only one available track. If the two signals in a column are identical, all tracks remain available for the bottom signal. On the contrary, the top signal can allocate the contact on any available track, regardless of whether the two signals in a column are distinct or not.

The layout style in [11] prioritizes routing simplicity over strict adherence to transistor width specifications. Transistor widths are treated as flexible and adjusted as needed to facilitate easier routing. For example, the DFFHQN cell in [11] does not follow the transistor width specification. Figs. 3(d) to 3(g) demonstrate why the cell design complying with transistor width specification is much harder than that in [11]. The layout style in Figs. 3(d) and 3(f) are ours, while those in Figs. 3(e) and 3(g) are from previous work [11]. Fig. 3(d) shows that the layout of two non-overlapping devices with small widths requires two M0 and one M1 wires to realize signal *Y*'s connection. M0-merge cannot be used in this case. In contrast, Fig. 3(e) demonstrates the flexible MOS widths allow the use of an M0-merge for routing signal *Y*. The assumption in [11] that two identical signals in the same column can always be routed with an M0-merge may fail under real MOS width specification.

In Fig. 3(f), top devices occupy the top three tracks, leaving only one bottom free track for the bottom signals, which produces a design rule violation (DRV) between signal *S* with two other M0 wires during M0 access. In Fig. 3(g), the routing model in [11] encourages signal *S* accessing the top M0 track, which results in misalignment between two diffusions in the top device; however, they should be aligned.

2.3 BS Routing Resources

BS routing resources, which are manufactured separately, are located under the substrate, while FS routing resources are on the substrate. Therefore, the metal pitch, metal width, and design rules for the BS are independent of the FS. To connect to the frontside metals (FSMs), specifically M0 in this work, BSMs utilize nTSVs that pass through the substrate. Careful placement of these nTSVs is crucial to avoid damaging any existing devices, such as poly or diffusion layers. In this work, we assume nTSVs can be placed on any track and in any column as long as there is no overlap with devices in the *z*-direction. According to our assumption, an nTSV occupies one grid on the FS and retains its size before passing through the substrate. As the number of BSM layers is configurable, there are two BSM layers in this work: backside metal 1 (BSM1) and backside metal 2 (BSM2). We assume that the BSM1 pitch is twice the CPP, and its width equals three times the M1 width. There is one BSM2 routing track, with a width three times that of M2 metal and spacing rules the same as M2. Fig. 4 shows a cross-sectional view of the layers on front and back sides.

2.4 Balanced Design-Rule-Check-Aware Remaining Pin Access (BDARPA)

The proposed BDARPA in [11] is an enhancement of remaining pin access (RPA) in [12]. BDARPA optimizes the RPA value of each IO pin evenly, while ensuring no violations occur when an M1 access point connects to an M2 wire during block-level routing. BDARPA provides a more accurate estimation of pin accessibility compared to RPA.

2.5 Graph-based Transistor Placement

In [13], there are four steps in transistor placement: clustering, pairing, chaining, and chain placement. Clustering uses common signals in PMOS and NMOS to divide the entire netlist into several sub-netlists. Pairing pairs the PMOS and NMOS with the same gate signal within the same clustering group. Chaining maximizes the number of shared diffusions using a depth-first search with a pruning technique. Finally, chain placement determines the permutation and orientation of the chains established in the previous step.

3. METHODOLOGY

The following discussion focuses on stacking P-type transistors on N-type transistors. The analysis can be extended to the configuration where N-type transistors are stacked on P-type transistors. Given a SPICE netlist, we first address CFET transistor folding and estimate the CPP lower bound of the folding results. The folding results with the lowest estimated value will proceed to the BS-resource-aware transistor placement stage. We utilize graph-based transistor placement from [13] to obtain placement results. Additionally, a diffusion column is considered undesirable if the top device's width in that column is not smaller than the number of routing tracks minus one. During the placement stage, we minimize CPP, FMRT, undesired diffusion columns, and then total wirelength. We also predict whether a column can accommodate an nTSV and calculate FMRT considering BS resources using a shortest path algorithm and DP. Finally, the SMT solver processes placement candidates, determining which tracks the devices occupy and completing cell routing.

3.1 CFET Transistor Folding

Transistor folding is the process of dividing a larger transistor into several smaller ones, called fingers. [11] briefly addresses CFET folding. Let tr_no_{M0} represent the number of M0 routing tracks. In [11], a transistor with a size greater than or equal to tr_no_{M0} is folded so that each finger has a size less than tr_no_{M0}, and the number of fingers is minimized. In our work, we not only fold transistors with sizes greater than or equal to tr_no_{M0}, but also fold top transistors with sizes greater than or equal to tr_no_{M0} minus one to explore more folding solutions. Since top devices block bottom devices, our goal is to create more room for the bottom by breaking down the top transistors. Additionally, the number of fingers in our approach ranges from $\lceil \frac{sz_{tran}}{tr\_no_{M0}} \rceil$ to $\lceil \frac{sz_{tran}}{x} \rceil$ (where sz_{tran} represents the transistor size, and x is user-defined; in this work, x equals two). Each finger size is equal to or less than tr_no_{M0}.

We construct a folding table by enumerating all possible finger configurations for each transistor and estimate the CPP lower bound for each folding result. When a transistor is folded into several fingers, its signal is split into the corresponding number of signals. We then sum the number of each diffusion signal across all transistors, as only diffusion signals affect subsequent estimations. A signal is considered odd when its count is odd. For example, two P-type transistors, (*net*1, *A*, *Y*) and (*VDD*, *B*, *net*1), are both folded into three fingers. The number of diffusion signals for *net*1, *Y*, and *VDD* is six, three, and three, respectively. Using Euler graphs, the following formula to compute the number of diffusion breaks (n_{df_bk}) is obtained:

$$n_{df\_bk} = \begin{cases} 0, \ if \ n_{o\_sg} < 3 \\ \left\lfloor \frac{n_{o\_sg}-1}{2} \right\rfloor, if \ n_{o\_sg} \geq 3 \end{cases}, \quad (1)$$

where n_{o_sg} is the number of odd signals. We sum the number of fingers and diffusion breaks to estimate the CPP lower bound. P-type and N-type devices are estimated separately, and we choose the larger estimated value as the CPP for the entire cell, as the larger value obviously dominates. We proceed with transistor placement only for the folding results that have the lowest estimated value in the folding table.

It is also worth mentioning that during the pairing stage, we exclude two illegal pairing results. One occurs when the NMOS width occupies all tracks and the *VDD* signal has no room to connect down to the power rail. The other occurs when the PMOS width occupies all tracks, leaving the NMOS signal no room to connect up to M0 if necessary.

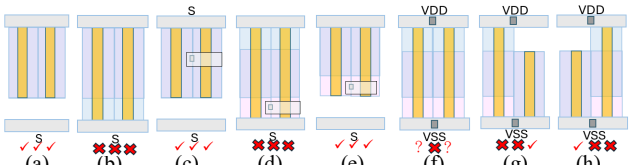


Figure 5. (a) - (e) n_{rm_trk} is 1, 0, 1, 0, and 1, respectively; (f) In case 4, n_{rm_trk} can only be used to predict the middlemost column, while the other two remain uncertain. (g) – (h) The left and right columns must be predicted by other abutting pairs. An nTSV can be placed by shifting the right (g) and left (h) devices, respectively.

3.2 BS-resource-aware MOS Placement

3.2.1 nTSV Column Prediction

After the pairing stage, we predict whether an nTSV can be placed on a column when abutting two transistor pairs. A local net refers to a net that is not *VDD*, *VSS*, or an IO signal and completes its routing without using metal, relying only on diffusion sharing.

We define the notations used in this section first. The number of remaining tracks on a column is n_{rm_trk}. wid_T and wid_B represent the larger width value of the two abutting top and bottom devices, respectively. n_{rm_trk} is calculated by subtracting the number of tracks that devices occupy on a column from the total number of M0 tracks, as placing an nTSV on devices will damage them. An n_{rm_trk} value greater than 0 implies that an nTSV can be placed on this column; otherwise, an nTSV cannot be placed.

In Fig. 5, a tick indicates that an nTSV can be placed in the column, a cross means the column cannot accommodate an nTSV, and a question mark represents uncertainty. We focus on discussing the three middle columns since the leftmost and rightmost columns must be predicted based on other abutting pairs.

Case 1: In Figs 5(a) and 5(b), the abutting bottom signal *S* is a local net. Since *S* does not need to connect up to M0, the bottom device can hide under the top device, potentially freeing up space for nTSV placement. In this case, $n_{rm\_trk} = tr\_no_{M0} -$

$max(wid_T, wid_B)$. In Fig. 5(b), there is no available column for placing an nTSV since n_{rm_trk} is 0.

Case 2: In Fig. 5(c), the abutting signals for top and bottom devices are the same (*S*), and the bottom *S* is not local. Even though S is not local, it also can use an M0-merge to connect to the top device, which then accesses M0. The bottom device can still hide under the top device. Here, $n_{rm\_trk} = tr\_no_{M0} - max(wid_T, wid_B)$.

Case 3: In Figs 5(d) and 5(e), the abutting signals for top and bottom devices differ, and the abutting bottom signal *S* is not local. Since *S* needs to allocate a contact to connect up to M0, at least one track must be reserved for contact allocation for *S*. In this case, $n_{rm\_trk} = tr\_no_{M0} - 1 - max(wid_T, wid_B - 1)$.

Case 4: In Figs 5(f) to 5(h), the abutting top and bottom signals are power and ground. Since both the power and ground signals need to connect to their respective power and ground rails, at least one top device and one bottom device must occupy the top and bottom tracks. n_{rm_trk} is calculated separately for both device types, with $n_{rm\_trk} = tr\_no_{M0} - wid_T - wid_B$.

To verify the accuracy of our nTSV column prediction, we also enumerate all possible track occupations of an MOS to check whether an nTSV can be placed on a column. We directly incorporate information about MOS track occupation in the following stages: pairing, chaining, and chain placement, which enlarges the placement solution space. Fig. 6 demonstrates the enumeration, pairing, and pairing abutment rules. In Fig. 6, a cross indicates an invalid pattern. Types 1 – 3 illustrate the partial enumeration with the top diffusion on different tracks. During pairing, the pairing pattern in Type 4 is invalid because the bottom device blocks the top device from connecting to the power rail, while Type 5 is valid. When abutting two transistor pairs, Type 6 is invalid as the top devices do not share their diffusion, while Type 7 is valid.

3.2.2 DP-based and BS-resource-aware FMRT

After the chaining and chain placement stages, we calculate the FMRT of a placement candidate (PC), considering BS resources to further reduce FMRT and ease M2 track usage in the subsequent routing stage. Fig. 7 depicts the routing results of two PCs, where Fig. 7(b) utilizes the BS resources while Fig. 7(a) cannot. Without considering BS resources, the evaluation of FMRT of two PCs are the same. However, after introducing BS resources, the FMRT in 7(b) becomes less than that in 7(a). Even though both consider BS resources, 7(b) further reduces FMRT by routing one signal using BS resources, while 7(a) cannot. Factors that impact FMRT when considering BS resources include the position of signals, and the number and position of nTSVs. We calculate the FMRT of a PC using a shortest path algorithm and DP.

For a PC, each signal has two ways to realize its shortest path. The first way uses only M0, while the second can utilize both M0 and BS resources. The shortest paths for a signal are determined in the absence of the shortest paths for other signals. All shortest paths are used in enumerating the FMRT. To identify the second shortest path, evaluating whether an nTSV can be placed on a column can help link the M0 and BS resources. Additionally, to ensure the BS resources are explored, their cost is set lower than that of the M0 resources. The path exploration space for each signal ranges from the leftmost to rightmost columns of its pins with slight expansion. The source and target for paths are set on the M0 layer.

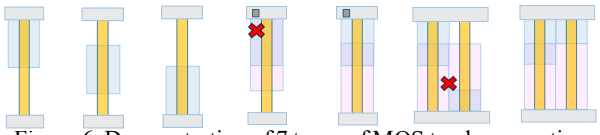


Figure 6. Demonstration of 7 types of MOS track occupations and the pairing and pair abutment rules, considering the information about MOS track occupation.

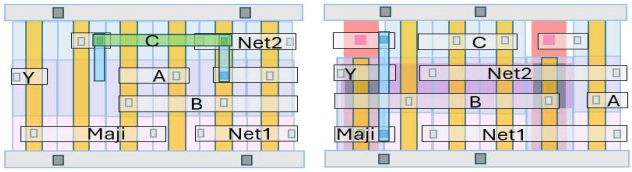


Figure 7. Different PCs in a cell. (a) uses one M2 track; (b) uses one BSM2 track and no M2 usage.

Algorithm 1. BS-resource-aware FMRT calculation

Input: a given transistor placement candidate *p*
Output: FMRT of the given candidate *p*
1. Construct graph $\mathcal{G}_\mathcal{R}$ of the candidate *p*
2. **for** *i*=1 to $2^{|V(\mathcal{G}_\mathcal{R})|}$
3. $dp[i] \leftarrow S$
4. $M \leftarrow \emptyset$
5. FMRT_Recursive_Process($\mathcal{G}_\mathcal{R}$, *M*)

Figure 8. Algorithm of BS-resource-aware FMRT.

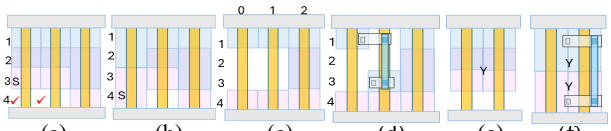
Algorithm 2. FMRT_Recursive_Process

Input: a graph $\mathcal{G}_\mathcal{R}$ and a node set *M*
Output: FMRT of the given graph $\mathcal{G}_\mathcal{R}$
1. **if** $\mathcal{G}_\mathcal{R}$ is $\emptyset$
2. **return** FMRT according to *M*
3. Pick a node *n* with the least node value and #neighbors
4. Push node *n* and its neighbors into queue *q*
5. **for** each node *s* in queue *q*
6. $\mathcal{G}_\mathcal{R}' \leftarrow \mathcal{G}_\mathcal{R}$ removes node *s*, its neighbors, and their edges
7. $M' \leftarrow M$ adds node *s*
8. $dp[\mathcal{G}_\mathcal{R}] \leftarrow$min($dp[\mathcal{G}_\mathcal{R}]$, FMRT_Recursive_Process($\mathcal{G}_\mathcal{R}'$, *M'*))
9. **return** $dp[\mathcal{G}_\mathcal{R}]$

Figure 9. FMRT_Recursive_Process.

After finding the paths, we construct a resource constraint graph, $\mathcal{G}_\mathcal{R}$, for the PC. If a signal uses BS resources in its second path, the signal has its respective node in $\mathcal{G}_\mathcal{R}$. If not, no node is created. If two signals' second paths overlap in the use of BS resources, an edge is created to link their respective nodes in $\mathcal{G}_\mathcal{R}$, indicating that the signals cannot use the BS resources simultaneously. We calculate the FMRT for each node in $\mathcal{G}_\mathcal{R}$ as its node value, assuming the corresponding signal uses its second path while the other signals choose their first paths for signal connections. The FMRT calculation focuses on the leftmost to rightmost columns of the corresponding signal's pins and equals the number of signal overlaps in the y-direction on M0.

DP is then applied to $\mathcal{G}_\mathcal{R}$ to determine which signals should utilize BS resources to minimize the FMRT of the PC. Fig. 8 and Fig. 9 illustrate our algorithm. In Fig. 8, we construct $\mathcal{G}_\mathcal{R}$, and $V(\mathcal{G}_\mathcal{R})$ represents the node set of $\mathcal{G}_\mathcal{R}$. There are $2^{|V(\mathcal{G}_\mathcal{R})|}$ possible combinations of BS signal usage. Each element of the array *dp* represents the FMRT when considering only the corresponding signals in the graph. We initialize all *dp* values to a large number *S* and set *M* as an empty node set. After initialization, a recursive procedure is called to calculate the FMRT of the PC. If $\mathcal{G}_\mathcal{R}$ is



(a) (b) (c) (d) (e) (f)

Figure 10. Device track occupation and SMT demonstration.

TABLE 1. NOTATIONS

S^{SN}/S^{N}	The subnet/net space
$TD_{i,t}/BD_{i,t}/G_{i,t}$	The bool variable indicates whether the i^{th} top diffusion/bottom diffusion/gate occupies track t.
$\mu_v^S(c,t,l,n)/\mu_e^S(c,t,l,n)$	The bool variable indicates whether the vertex/edge in c^{th} column, t^{th} track, l layer and S space is used by net n. A horizontal edge connects the c^{th} and $(c+1)^{th}$ columns. A vertical edge connects the t^{th} and $(t+1)^{th}$ tracks. A via edge connects the l^{th} and $(l+1)^{th}$ layers. S can be either SN or N.
$deg^{SN}(v)$	The integer variable indicates the number of used edges of vertex v in the SN space.

empty, we calculate and return the FMRT for the entire placement when the corresponding signals in M use their second paths, while other signals use their first paths for connections (Fig. 9, Lines 1-2).

We select a node n in $\mathcal{G}_{\mathcal{R}}$ with the smallest node value and the fewest neighbors to reduce recursion calls (Line 3). Node n and its neighbors are pushed into a queue, and we iterate through each node in the queue to call the recursive procedure, minimizing the dp value at each iteration (Lines 4-8). Node n and its neighbors are not selected simultaneously due to the overlap in BS resources. We iterate through all nodes in the queue to explore all possible solutions. Once a node s is selected, $\mathcal{G}_{\mathcal{R}}'$ ($\mathcal{G}_{\mathcal{R}}$ with node s, its neighbors, and their edges removed) and M' (M with node s added) are passed down to the next recursive level (Lines 6-8).

3.3 SMT-based Cell Routing

There are two important tasks for our SMT algorithm: determining the track occupation of devices and managing signal connections while considering BS resources, both of which are determined simultaneously. Due to the additional consideration of transistor width, which was simplified in previous work, the proposed SMT formulation needs to accurately handle device track occupation, diffusion sharing, diffusion merge, and contact allocation. The diffusion track occupation differs between Fig. 10(a) and Fig. 10(b), which shows that device track occupation impacts signal connections on both sides. For the FS, contacts can only be allocated on devices. Therefore, in Fig. 10(a), the signal S can only access M0 on the third track, whereas in Fig. 10(b), it can access both the third and fourth tracks. For the BS, nTSVs cannot overlap with devices in the z-direction. As a result, in Fig. 10(a), there are two ticks indicating positions where an nTSV can be placed, whereas in Fig. 10(b), no such ticks are present.

Table 1 lists the notations used in the SMT formulations. When a net is a multiple-pin net, we decompose it into several two-pin nets, arranged from the leftmost to the rightmost column. Even if the top and bottom signals in a column are the same, we still use a two-pin net to represent them. The solution space for a two-pin net ranges from its leftmost to rightmost columns, with slight expansion. We connect each two-pin net in the subnet space, and the routing results in the subnet space are projected onto the net space. To expedite the process, we first define a net order, which is sorted in descending order based on the signal span length. All nets are routed using an M0 pattern in the initial iteration, and one more net is routed using multi-commodity flow (MCF) in subsequent iterations until the routing is complete. T represents the M0 routing track set, and $|T|$ is four in this work. Our optimization objectives, in lexicographic order, are to minimize the number of M2 tracks, maximize BDARPA as defined in [11], and minimize the total wirelength.

3.3.1 MOS Constraints

wid represents the number of tracks that the diffusion of a device occupies in a column, with one set of *wid* continuous variables being true for the device. We enumerate all possible track occupations for the diffusion. Eq. (2) defines the diffusion of the i^{th} top device, with its width being two. Other devices and widths are treated similarly. In Fig. 10(c), the encoded result for the top diffusion number zero/two is {true, false, false, false}/{true, true, true, false}.

$$(TD_{i,0} \wedge TD_{i,1} \wedge \sim TD_{i,2} \wedge \sim TD_{i,3}) \vee (\sim TD_{i,0} \wedge TD_{i,1} \wedge TD_{i,2} \wedge \sim TD_{i,3}) \vee (\sim TD_{i,0} \wedge \sim TD_{i,1} \wedge TD_{i,2} \wedge TD_{i,3}) \quad (2)$$

In a device, if the diffusion occupies a track, the gate will also occupy the corresponding track in its column. We represent i^{th} gate using Eq. (3). Both the top and bottom devices are considered due to the same gate signal. Track occupation for the gate is also enumerated.

$$(TD_{i,t} \rightarrow G_{i,t}) \wedge (BD_{i,t} \rightarrow G_{i,t}) \quad (3)$$

3.3.2 Connectivity Constraints

M0, M2, and BSM2 are used for horizontal routing, while M1 and BSM1 are used for vertical routing. Eq. (4) defines a horizontal edge, with vertical and via edges treated similarly.

$$\mu_e^{SN}(c,t,l,n) \rightarrow \mu_v^{SN}(c,t,l,n) \wedge \mu_v^{SN}(c+1,t,l,n) \quad (4)$$

$S^{SN(a,b)}$ represents a two-pin net connecting pins a and b in the subnet space. Using the MCF, we restrict the degree of vertices in Eq. (5) and Eq. (6) to complete the two-pin net connection. In Eq. (5), the pin may be a floating pin. V is the vertex set of $S^{SN(a,b)}$.

$$\wedge_{p=a,b}(\sum_{\{v|v\in p\}} deg^{SN(a,b)}(v) == 1) \quad (5)$$

$$\wedge_{v\in V, v\notin a, v\notin b}\left(deg^{SN(a,b)}(v) == 0 \vee deg^{SN(a,b)}(v) == 2\right) \quad (6)$$

3.3.3 Diffusion Sharing Constraints

There must be at least one track abutment between the diffusions of adjacent devices, or a two-pin net can be used to connect them. Figs. 10(c) and 10(d) illustrate these two options. Eq. (7) defines the first option for the connection between the i^{th} and $(i+1)^{th}$ top devices.

$$\vee_{t\in T} TD_{i,t} \wedge TD_{i+1,t} \quad (7)$$

3.3.4 Diffusion Merging Constraints

There must be at least one track overlap in the z-direction between the P-type and N-type diffusions (using an M0-merge to connect them), or a two-pin net can achieve connection. Figs. 10(e) and 10(f) illustrate two possible options for connecting the top and bottom signal Y. Eq. (8) defines the first option to connect the two types of diffusion between the i^{th} and $(i+1)^{th}$ devices.

$$\vee_{t\in T}\left(TD_{i,t} \vee TD_{i+1,t}\right) \wedge \left(BD_{i,t} \vee BD_{i+1,t}\right) \quad (8)$$

3.3.5 Contact Allocation Constraints

Contacts can only be allocated on devices. Additionally, when the two signals in a column differ, the contact allocation for the bottom signal will be blocked by the top signal. However, when the two signals are identical, the tracks occupied by two signals are available. Eq. (9) to Eq. (11) represent the contact allocation for the diffusion column. For the bottom diffusion, Eq. (9) and Eq. (10) represent the cases where the two signals in a column are different and identical, respectively. Eq. (11) defines for the top diffusion when the two signals in a column are different.

A top/bottom diffusion column consists of its two corresponding top/bottom devices due to diffusion sharing. i and j are the indexes of the corresponding devices for c^{th} diffusion column. The top/bottom signal in the c^{th} column is denoted as $TS(c)/BS(c)$. CA represents the contact layer.

$$\mu_v^N(c,t,CA,BS(c)) \rightarrow \sim(TD_{i,t} \vee TD_{j,t}) \wedge (BD_{i,t} \vee BD_{j,t}) \quad (9)$$

$$\mu_v^N(c,t,CA,BS(c)) \rightarrow TD_{i,t} \vee TD_{j,t} \vee BD_{i,t} \vee BD_{j,t} \quad (10)$$

$$\mu_v^N(c,t,CA,TS(c)) \rightarrow TD_{i,t} \vee TD_{j,t} \quad (11)$$

3.3.6 nTSV Constraints

An nTSV cannot overlap with devices in the z-direction. Eq. (12) applies to the diffusion, and the gate is treated similarly. $nTSV$ represents the nTSV layer. i and j are the indexes of the corresponding devices for c^{th} diffusion column.

$$\mu_v^N(c,t,nTSV,n) \rightarrow \sim(TD_{i,t} \vee TD_{j,t} \vee BD_{i,t} \vee BD_{j,t}) \quad (12)$$

4. EXPERIMENTAL RESULTS

This work is implemented in C++ and executed on a Linux platform with a 2.1 GHz twenty-core CPU and 252 GB of memory. We use the Z3 SMT solver [14] for the routing tasks. Our synthesis input is the ASAP7 SPICE netlist [15], and the cell structure is based on a 4.5T layout with buried power rails.

4.1 Cell Quality Comparison

All our cell libraries strictly adhere to the SPICE netlists. The design rules follow those in [11], with MAR/EOL/VR/PRL/SHR values set to 1/2/1/1/2. For transistor folding, F:CFET represents a library that considers the CFET stacked structure, while F:[11] represents a library that folds the way [11] does. Regarding routing resources, R:FB refers to a library that takes both FS and BS resources into account, while R:F refers to a library that only considers FS resources. For the nTSV column, N:P represents a library using the prediction method proposed in this work, while N:E represents a library that enumerates all possible device track occupations.

In Table 2, [11] successfully routes all cells. For each cell type, we count how many cells have a #CPP that is not larger than [11] (#CPP≤[11]), both #CPP and #M2 tracks that are not larger than [11] (#CPP≤[11]&#M2 T≤[11]), a #CPP that is lower than [11] (#CPP<[11]), and a #CPP that is not larger but with #M2 tracks lower than [11] (#CPP≤[11]&#M2 T<[11]). Our folding method benefits cells in the types INV BUF, AND OR, NAND NOR, AO AOI, OA OAI, and XOR XNOR. Due to our folding method, there are three/two/four cells with a lower #CPP than [11] in the first library for INV BUF/AND OR/NAND NOR. More than half (eleven out of twenty) of the OA OAI cells, and all XOR XNOR cells, have a #CPP that is not larger than [11]. BS resources benefit cells in the NAND NOR, AO AOI, OA OAI, MAJ MAJI, XOR XNOR, and DFF Latch types. Due to BS resources, in the first library, seven/ten cells have lower #CPP/#M2 tracks, two cells have lower #M2 tracks, and all cells have the same #CPP and #M2 tracks in AO AOI, MAJ MAJI, and DFF Latch types, respectively.

The reason for the larger #CPP in the OA OAI type is that [11] simplifies the routing model, and half of our OA OAI cells encounter the scenario shown in Fig. 3(f). Compared to [11], our library (F:CFET+R:FB+N:P in Table 2) contains six cells with one additional #CPP and three cells with two additional #CPP in the OA OAI type. Fig. 11 illustrates our DFFHQN layout with BS resources.

TABLE 2. LIBRARY COMPARISION AMONG [11] AND OURS.

Cell type/ #Cell	#CPP<=[11]/ #CPP<=[11]&#M2T<=[11]/ #CPP<[11]/ #CPP<=[11]&#M2 T<[11]			
	F:CFET+R: FB+N:P	F:CFET+ R:F+N:P	F:[11]+R :FB+N:P	F:[11]+R :F+N:P
INV BUF/12	12/12/3/0	12/12/3/0	12/12/0/0	12/12/0/0
AND OR/21	21/21/2/0	21/21/2/0	21/21/0/0	21/21/0/0
NAND NOR/23	23/23/4/0	23/23/4/0	21/21/0/0	21/20/0/0
AO AOI/36	36/36/7/10	33/31/0/0	35/35/7/10	32/30/0/0
OA OAI/20	11/10/0/0	11/8/0/0	9/8/0/0	9/7/0/0
MAJ MAJI/3	3/3/0/2	3/3/0/0	3/3/0/2	3/3/0/0
XOR XNOR/6	6/5/0/2	6/4/0/2	5/4/0/2	5/3/0/0
HA FA/2	2/2/0/0	2/2/0/0	2/2/0/0	2/2/0/0
DFF Latch/14	14/14/0/0	14/6/0/0	14/14/0/0	14/6/0/0

TABLE 3. NTSV PREDICTION, CELL QUALITY, AND RUNTIME COMPARISON.

Cell type/ #Cell	#CPP/ #M2 tracks/ Avg. Time		
	F:CFET+ R:FB+N:P	F:CFET+ R:FB+N:E	[11]
INV BUF/12	120/0/1.7	120/0/28.1	123/0/6.5
AND OR/21	194/1/4.7	194/1/16.4	196/1/4.5
NAND NOR/23	192/0/4.2	192/0/23.9	200/0/4.7
AO AOI/36	320/0/14.5	320/0/48.5	327/10/12.3
OA OAI/20	169/5/9.1	169/5/1317.7	157/1/3.8
MAJ MAJI/3	26/0/2.6	26/0/3.0	26/2/4.9
XOR XNOR/6	58/3/4.8	58/3/266.0	58/4/4.8
HA FA/2	22/2/11.0	22/2/13469.1	22/2/27.4
DFF Latch/14	208/0/14.9	208/0/1845.3	208/0/14.3
Sum	1309/11/67.4	1309/11/17018.2	1317/20/83.4

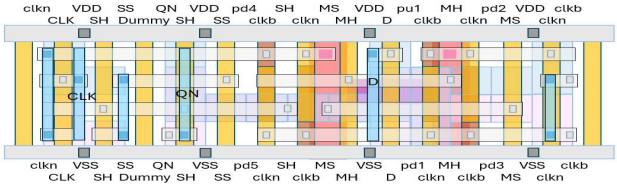


Figure 11. Our DFFHQN layout with BS resources.

4.2 NTSV Column Prediction, Cell Quality and Runtime Comparison

In Table 3, we sum the #CPP and #M2 tracks and calculate the average runtime for each cell type, with time measured in seconds. The first library predicts the nTSV columns, while the second enumerates all device track occupations to obtain the actual nTSV columns. The quality of both methods is the same, but the prediction method is 252 times faster on average than the enumeration method, reducing total runtime from 17018 seconds to 67 seconds. With total #CPP decreasing from 1317 to 1309, #M2 tracks dropping from 20 to 11, and the average runtime improving from 83.4 to 67.4 seconds, we achieve a 1%/45%/19% reduction in #CPP/#M2 tracks/runtime compared to [11]. Excluding the OA OAI type, we achieve an even greater reduction of 2% in #CPP and 68% in #M2 tracks.

5. CONCLUSIONS

To reduce the cell area and #M2 tracks, this is the first work to introduce BS resources and address stacked-structure transistor folding during the CFET cell synthesis. We fully adhere to SPICE netlists, while all previous works do not. Euler paths are utilized to estimate the CPP lower bound for folding results. In the placement stage, we calculate the FMRT with BS resources using DP. Finally, SMT decides MOS track occupation and completes cell routing. Compared to the state-of-the-art [11], we achieve superior results with reductions of 1%, 45%, and 19% in #CPP, #M2 tracks, and runtime, respectively.

6. REFERENCES

[1] G. Sisto et al. "IR-Drop Analysis of Hybrid Bonded 3D-ICs with Backside Power Delivery and μ- & n- TSVs." 2021 IEEE International Interconnect Technology Conference, pp. 1-3, 2021.

[2] R. Chen et al. "Design and Optimization of SRAM Macro and Logic Using Backside Interconnects at 2nm node." 2021 IEEE International Electron Devices Meeting (IEDM), pp. 22.4.1-22.4.4, 2021.

[3] K. Baek et al. "Simultaneous Transistor Folding and Placement in Standard Cell Layout Synthesis." 2021 IEEE/ACM International Conference On Computer Aided Design, pp. 1-8, 2021.

[4] A. Lu et al. "Simultaneous transistor pairing and placement for CMOS standard cells." 2015 Design, Automation & Test in Europe Conference & Exhibition, pp. 1647-1652, 2015.

[5] P. V. Cleeff et al. "BonnCell: Automatic Cell Layout in the 7-nm Era." IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, vol. 39, Issue 10, pp. 2872-2885, 2020.

[6] C.-Y. Hwang et al. "A fast transistor-chaining algorithm for CMOS cell layout." IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, vol. 9, Issue 7, pp. 781-786, 1990.

[7] Y.-L. Li et al. "NCTUcell: A DDA-aware cell library generator for FinFET structure with implicitly adjustable grid map." Proceedings of the 56th Annual Design Automation Conference 2019, pp. 1-6, 2019.

[8] N. Ryzhenko et al. "Pin Access-Driven Design Rule Clean and DFM Optimized Routing of Standard Cells under Boolean Constraints." Proceedings of the 2019 International Symposium on Physical Design, pp. 41-47, 2019.

[9] B.-X. Song et al. "Routability Booster – Synthesize a Routing Friendly Standard Cell Library by Relaxing BEOL Resources." Proceedings of the 2024 International Symposium on Physical Design, pp. 185-193, 2024.

[10] C.-K. Cheng et al. "Complementary-FET (CFET) standard cell synthesis framework for design and system technology co-optimization using SMT." IEEE Transactions on Very Large-Scale Integration Systems, vol. 29, Issue 6, pp. 1178-1191, 2021.

[11] T.-W. Lee et al. "Scalable CFET Cell Library Synthesis with A DRC-Aware Lookup Table to Optimize Valid Pin Access." Proceedings of the 2025 International Symposium on Physical Design, 2025.

[12] J. Seo et al. "Pin accessibility-driven cell layout redesign and placement optimization." Proceedings of the 54th Annual Design Automation Conference, pp. 1-6, 2017.

[13] Y.-C. Hsich et al. "LiB: A CMOS cell compiler." IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, vol. 10, Issue 8, pp. 994-1005, 1991.

[14] L. D. Moura et al. "Z3: An efficient SMT solver." Tools and Algorithms for the Construction and Analysis of Systems, pp. 337-340, 2008.

[15] L. T. Clark et al. "ASAP7: A 7-nm finFET predictive process design kit." Microelectronics Journal, vol. 53, pp. 105-115, 2016.